

DAT62-4

Computer Vision

From image tensors to deployed, interpretable vision systems

A computer vision system is only as good as the pixels you feed it, the invariances you teach it, and the failure modes you anticipate.

Albert School — 22 hours

1 Preface

In *Neural Networks* (DAT62-2) you stopped re-explaining theory and started building: you implemented feedforward networks and CNNs in PyTorch, drove training loops with autograd, applied transfer learning with ResNet and EfficientNet, and learned to read loss and accuracy curves to diagnose overfitting, underfitting, and vanishing gradients. You finished by defending an architecture choice in a model card. That practitioner’s reflex — build, train, measure, defend — is exactly what this course assumes.

Computer Vision takes those same tools and points them at a single applied domain end to end. The mathematics is almost entirely behind us; what remains is applied skill. We start one step before the model — at the *pixels* — because in vision the most expensive mistakes happen there: a wrong color space, a missed normalization, an augmentation that destroys the very signal you needed. From there we build a CNN classifier from scratch, adapt pretrained backbones, choose metrics that survive class imbalance and asymmetric business cost, detect objects with bounding boxes, and learn the vocabulary that separates classification from detection from segmentation.

The second half of the course is about *fragility*. Real vision systems fail in production for reasons that never show up on a clean test set: distribution shift, label noise, long-tail classes. You will learn to diagnose each and propose a mitigation. You will open the black box with Grad-CAM to see which pixels actually drove a prediction — and learn to explain that to a stakeholder who has never heard of a gradient. Finally you will assemble the whole arc into a complete project with a model card.

Throughout, each idea arrives with a concrete example before any abstraction, common mistakes are flagged where students reliably stumble, and figures appear wherever a picture genuinely helps. We build on DAT62-2 rather than repeating it: if “convolution,” “fine-tuning,” or “model card” is unfamiliar, revisit that course first.

Contents

1	Preface	2
2	How an image becomes a tensor	4
2.1	A motivating example	4
2.2	Channels, resolution, and shape	4
2.3	Color space	5
2.4	Normalization	5
3	Data augmentation	6
3.1	The problem augmentation solves	6
3.2	One transformation, one invariance	6
3.3	The justification discipline	6
4	Building a CNN from scratch	7
4.1	Why not just use a fully connected network?	7
4.2	The convolutional layer	7
4.3	Pooling	7
4.4	The fully connected head	8
5	Standing on giants: pretrained models	8
5.1	Why train from scratch is usually the wrong default	8
5.2	Two ways to reuse a backbone	9
5.3	Choosing, and justifying, between them	9
6	Evaluation under real-world constraints	10
6.1	Why accuracy is the wrong default metric	10

6.2	The confusion matrix and the metrics built on it	10
6.3	Letting business cost choose the operating point	10
7	Object detection	11
7.1	From “what” to “what and where”	11
7.2	IoU: measuring how good a box is	11
7.3	Non-maximum suppression	12
8	A map of vision tasks	12
8.1	Three questions, three tasks	12
8.2	One business use case each	13
9	Why production vision systems fail	13
9.1	Clean test scores lie	13
9.2	Distribution shift	13
9.3	Label noise	13
9.4	Long-tail class imbalance	14
10	Looking inside the model: Grad-CAM	14
10.1	Why a number is not enough	14
10.2	What Grad-CAM shows	15
10.3	Using it, and communicating it	15
11	Delivering a complete vision project	15
11.1	The arc, end to end	15
11.2	The model card	16

2 How an image becomes a tensor

2.1 A motivating example

Open a photo of a cat on your phone. To you it is a cat; to a neural network it is a grid of numbers. Before any model can learn anything, that visual scene must be turned into a tensor — a multi-dimensional array of numbers — and *how* you do that conversion silently sets a ceiling on everything downstream. Two engineers handed the same dataset can get wildly different accuracy purely because one normalized the pixels and the other did not.

Let us make the grid concrete. A small grayscale image is just a 2-D array where each entry is a brightness value, conventionally an 8-bit integer from 0 (black) to 255 (white).

15	40	90
60	200	210
120	230	250

one channel: brightness $\in [0, 255]$

Figure 1: A 3×3 grayscale patch. The picture *is* the matrix of numbers; the visual brightness of each cell is exactly its stored value scaled to $[0, 1]$.

2.2 Channels, resolution, and shape

A color image adds a third axis: the **channel**. In the standard RGB color space each pixel stores three numbers — how much red, green, and blue light it emits — so the image is a tensor of shape (height, width, channels), for example $224 \times 224 \times 3$.

Common pitfall PyTorch’s vision stack expects **channels-first** order — shape (C, H, W) , e.g. $(3, 224, 224)$ — while libraries like NumPy, PIL, and OpenCV hand you **channels-last** (H, W, C) . Worse, OpenCV loads channels in **BGR** order, not RGB. A model fed BGR when it was trained on RGB will not crash — it will just quietly lose accuracy because every “red” feature it learned now sees blue. Mismatched channel order and axis order are the two most common silent bugs in a vision pipeline.

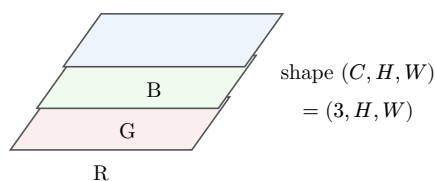


Figure 2: An RGB image is three stacked 2-D planes — one per color channel. Stacking them along the channel axis gives the (C, H, W) tensor a CNN consumes.

Resolution is the height $\times$ width in pixels. It is a direct lever on the accuracy/cost trade-off. Higher resolution preserves fine detail (the texture that distinguishes a husky from a wolf) but the compute and memory of a convolutional layer scale with the number of pixels, so doubling resolution roughly quadruples the cost of early layers. Downsizing too aggressively can erase the very feature the task depends on.

Worked example. A model that grades the ripeness of fruit from color needs little resolution — 64×64 may suffice. A model that reads a serial number stamped into metal needs enough pixels that each digit survives downsizing; at 64×64 the digits blur into noise and accuracy collapses no matter how good the architecture. The right resolution is set by the smallest feature the task depends on, not by a default.

2.3 Color space

RGB is the default, but it is not the only choice, and sometimes not the best one. A **color space** is a coordinate system for color. Converting to grayscale collapses the three channels into one, discarding hue — sensible when color is irrelevant (reading text) and harmful when it is the signal (detecting rust). Other spaces such as HSV separate hue, saturation, and brightness onto their own axes, which can make a model more robust to lighting because brightness is isolated on a single channel.

The practical rule: keep the channels that carry task signal, drop the ones that only add noise and cost. If color cannot possibly matter for your task, grayscale halves your input size for free.

2.4 Normalization

Raw pixel values live in $[0, 255]$. Feeding those directly into a network is a classic mistake: large, all-positive inputs push activations into saturated regions and make the loss surface badly scaled, so training is slow and unstable. **Normalization** rescales pixels to a small, centered range.

The two-step recipe almost everyone uses: first scale to $[0, 1]$ by dividing by 255, then standardize each channel to roughly zero mean and unit variance by subtracting a per-channel mean μ_c and dividing by a per-channel standard deviation σ_c :

$$x'_c = \frac{x_c/255 - \mu_c}{\sigma_c}.$$

Common pitfall When you use a pretrained backbone you must normalize with the **exact mean and standard deviation the backbone was trained with** — for ImageNet models these are the well-known constants $\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$. Using your own dataset's statistics, or skipping normalization, shifts every input away from what the frozen filters expect and silently degrades accuracy. This is the single most common transfer-learning bug.

A second non-negotiable rule: compute normalization statistics on the **training set only**, then apply those same fixed numbers to validation and test data. Computing them over the whole dataset leaks information from the test set into training — a small but real form of data leakage.

Chapter summary. An image is a tensor of shape (C, H, W) . Resolution trades detail against compute; color space decides which channels carry signal; normalization (scale to $[0, 1]$, then standardize per channel) makes training stable — and with a pretrained model you must reuse *its* statistics. Watch the two silent bugs: channel order (RGB vs BGR) and axis order (channels-first vs channels-last).

3 Data augmentation

3.1 The problem augmentation solves

You have 2{,}000 labeled photos of cats and dogs. Your CNN reaches 99% on them and 74% on new photos — a textbook case of overfitting (DAT62-2): the model memorized *these* images instead of learning what a cat looks like. You could collect more data, but labeling is expensive. **Data augmentation** manufactures new training examples on the fly by applying label-preserving transformations to the images you already have.

The deep idea is not “more data for free.” It is that **each transformation teaches the model an invariance** — a statement of the form “the label does not change when I do this to the image.” Every augmentation you choose is a claim about your problem, and a wrong claim hurts.

3.2 One transformation, one invariance

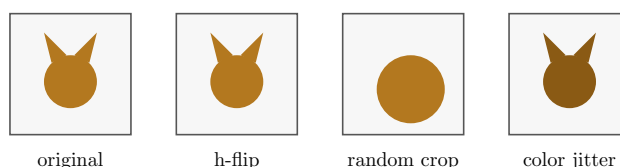


Figure 3: Four label-preserving views of the same “cat.” Each transformation asserts an invariance: left–right symmetry, position/scale, and lighting do not change the class.

Horizontal flip teaches left–right invariance: a cat facing left is still a cat. This is almost always safe for natural objects. **Random crop** (take a random sub-rectangle and resize it back) teaches that the object’s position and scale do not matter, and forces the model to recognize a cat from a part of it. **Rotation** teaches orientation invariance over a small angular range. **Color jitter** (randomly perturb brightness, contrast, saturation, hue) teaches robustness to lighting and camera differences.

3.3 The justification discipline

Common pitfall Augmentation is not a bag of tricks to apply blindly — the transformation must preserve the label. A horizontal flip turns the digit “2” into a meaningless mirror shape, and a 180° rotation turns “6” into “9”: for handwritten-digit recognition both augmentations *destroy* the label and will train the model to be wrong. The same flip is perfectly safe for cats. The transformation is only valid if the invariance it asserts is true for *your* task.

The professional habit is to justify every transformation out loud. “I add horizontal flip because a product photographed from the left is the same product from the right. I add color jitter because our photos come from many phone cameras under different lighting. I do *not* add vertical flip because our products are always upright and an upside-down image never occurs in deployment.” That last sentence matters: an augmentation that creates inputs you will never see at deployment wastes capacity teaching an irrelevant invariance.

Worked example. A medical model classifies skin lesions. Horizontal and vertical flips and small rotations are all justified — a lesion has no preferred orientation. But aggressive color jitter is dangerous: the *color* of a lesion is diagnostic signal, so perturbing hue could teach the model to ignore the very feature a dermatologist relies on. Here the same color jitter that helped with cats would actively harm the task.

A few practical rules close the chapter. Augmentation is applied to the **training set only**, freshly randomized each epoch, so the model effectively never sees the same image twice; validation and test data are left untouched so your metric reflects real performance. And augmentation *complements* more data, it does not replace it — it cannot invent a class or a viewpoint that is entirely absent from your originals.

4 Building a CNN from scratch

4.1 Why not just use a fully connected network?

You already know how to build a feedforward network. Why not flatten the $224 \times 224 \times 3$ image into a vector of 150{,}528 numbers and feed a dense layer? Two reasons make this hopeless for images. First, the parameter count explodes: a single dense layer with 1{,}000 units would need 150 million weights. Second, and deeper, a dense layer has no notion of **locality** — it treats the top-left pixel and the bottom-right pixel as unrelated inputs, throwing away the fact that nearby pixels form edges, and that an edge means the same thing wherever it appears. The convolutional layer is the architecture built to exploit exactly that spatial structure.

4.2 The convolutional layer

A convolutional layer slides a small learnable filter (a **kernel**, say 3×3) across the image. At each position it computes a weighted sum of the pixels under it, producing one number; sweeping across the whole image yields a **feature map** that highlights wherever the filter's pattern occurred.

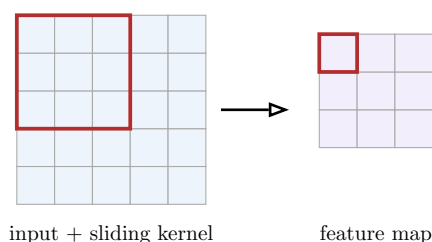


Figure 4: A 3×3 kernel slides over a 5×5 input. Each placement produces one feature-map value; the red cell on the right is the output for the red kernel position on the left.

Three properties make this powerful. **Parameter sharing**: the same nine kernel weights are reused at every position, so the layer has very few parameters and learns a pattern detector that works anywhere in the image. **Local connectivity**: each output depends only on a small neighborhood, matching how visual features are local. **Translation equivariance**: if the input pattern shifts, its activation shifts the same way — an edge detector fires on edges wherever they are.

A layer learns *many* kernels in parallel; each produces its own feature map, so a layer with 32 filters turns its input into a stack of 32 feature maps. Early layers learn generic detectors — edges, color blobs — and later layers, fed by earlier ones, compose these into textures, parts, and eventually whole-object detectors. This hierarchy is learned, not designed.

4.3 Pooling

After a convolution we usually **pool**: downsample each feature map by summarizing small neighborhoods. **Max pooling** over a 2×2 window keeps only the largest value in each window, halving height and width.

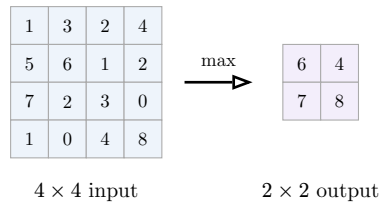


Figure 5: Max pooling with a 2×2 window keeps the maximum of each quadrant, shrinking the map fourfold while retaining the strongest response.

Pooling does two jobs. It **shrinks** the feature maps, cutting compute and memory for later layers, and it grants a small amount of **translation invariance**: if the strongest activation moves by one pixel, the pooled output is often unchanged. The stack of convolution-then-pool also makes the **receptive field** — the region of the original image that influences one deep activation — grow with depth, so deep units “see” large parts of the image even though every kernel is tiny.

4.4 The fully connected head

Convolution and pooling produce a stack of small feature maps — a compact description of *what* is in the image and roughly *where*. To turn that into a class decision we flatten it and pass it through one or two **fully connected** (dense) layers ending in one output per class. A softmax converts those outputs into class probabilities, and we train against the cross-entropy loss exactly as in DAT62-2.

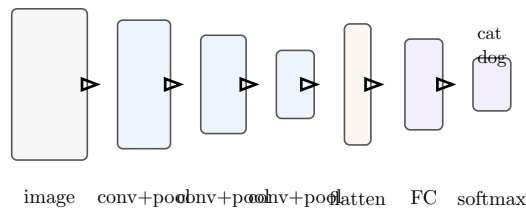


Figure 6: A CNN pipeline. Convolution+pooling blocks build an increasingly abstract, increasingly small feature representation; the fully connected head reads it out into class scores.

How to read the three layer types. *Convolutional* layers detect local patterns and build a what-and-where feature hierarchy through parameter sharing. *Pooling* layers downsample, cutting cost and adding small translation invariance. *Fully connected* layers at the head combine the global evidence into a class decision. Convolutions extract features; the dense head classifies them.

5 Standing on giants: pretrained models

5.1 Why train from scratch is usually the wrong default

You can now build a CNN, but should you? Training a competitive image model from random weights needs millions of labeled images and days of GPU time. The practitioner’s default — which you met in DAT62-2 — is **transfer learning**: take a network already trained on a huge dataset (ImageNet, 1.2 million images, 1,000 classes) and reuse it. The intuition is that the early and middle layers have already learned generic visual features — edges, textures, shapes — that transfer to almost any vision task.

Two backbones dominate. **ResNet** introduced *residual connections* (skip connections that let gradients flow past many layers), which made very deep networks trainable; it is a reliable, well-understood default. **EfficientNet** uses a principled rule for jointly scaling depth, width,

and resolution, reaching similar or better accuracy with fewer parameters and less compute — attractive when you are deployment-constrained. For most projects either is a fine starting point; the choice between them is usually about your latency and memory budget, not raw accuracy.

5.2 Two ways to reuse a backbone

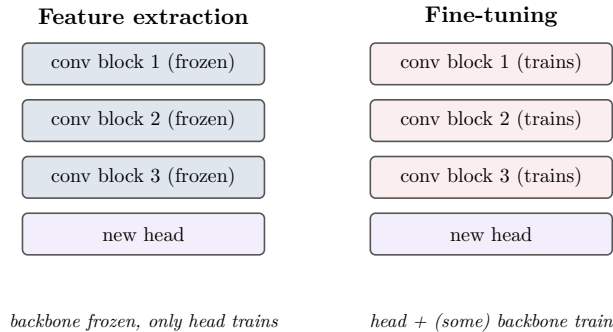


Figure 7: Two ways to reuse a pretrained backbone. Feature extraction freezes the backbone and trains only a new head; fine-tuning unfreezes some or all backbone layers and continues training them at a small learning rate.

Feature extraction: freeze the entire backbone and treat it as a fixed function that maps an image to a feature vector; train only a new classification head on top. This is fast, needs little data, and resists overfitting because almost nothing is being learned — but it cannot adapt the features themselves, so it underperforms when your images look unlike ImageNet’s.

Fine-tuning: unfreeze some (often the later) or all backbone layers and continue training them on your data, usually at a *small* learning rate so you nudge the pretrained weights rather than destroying them. This adapts the features to your domain and reaches higher accuracy — at the cost of more data, more compute, and more overfitting risk.

5.3 Choosing, and justifying, between them

The choice follows two axes: how much labeled data you have, and how similar your images are to ImageNet.

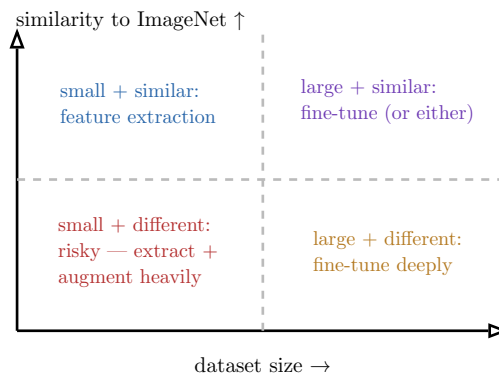


Figure 8: A decision map for transfer learning. Little data favors freezing more (feature extraction); a big domain gap favors training more layers. The hard corner is little data plus a large domain gap.

Worked example. *Task A:* classify house cats by breed from clear pet photos. These look much like ImageNet and you have only 1{,}500 images, so **feature extraction** is the right call — freeze ResNet, train a small head, done in minutes. *Task B:* classify defects in grayscale X-ray welds, with 50{,}000 labeled images that look nothing like natural photos. Here **fine-**

tuning deeper into the backbone is justified: you have the data to adapt the features and the domain gap means the off-the-shelf features are a poor fit. The justification — data volume and domain similarity — is exactly what a model card must record.

Common pitfall Two fine-tuning mistakes recur. First, using a *large* learning rate on an unfrozen backbone wipes out the pretrained knowledge in the first few batches (“catastrophic forgetting”) — fine-tune at roughly a tenth of a from-scratch rate or less. Second, fine-tuning a big backbone on a tiny dataset overfits almost instantly; if you have little data, freeze more and lean on augmentation.

6 Evaluation under real-world constraints

6.1 Why accuracy is the wrong default metric

A fraud-image detector is trained on a dataset that is 99% “clean” and 1% “fraudulent.” A model that predicts “clean” for *every* image scores 99% accuracy — and is completely useless, because it never catches the thing you built it for. The moment classes are imbalanced or mistakes cost different amounts, accuracy stops measuring what you care about. Choosing the metric is itself a modeling decision, and in this course we make it deliberately.

6.2 The confusion matrix and the metrics built on it

Every metric for a binary classifier is built from four counts: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).

	pred +	pred −
actual +	TP	FN
actual −	FP	TN

Figure 9: The confusion matrix. The two off-diagonal cells are the two kinds of error — and they usually do not cost the same.

From these we define:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Precision answers “of the images I flagged positive, how many really were?” — it punishes false alarms. **Recall** answers “of the truly positive images, how many did I catch?” — it punishes misses. They trade off: lowering the decision threshold catches more positives (higher recall) but admits more false alarms (lower precision). The **F1 score**, their harmonic mean $2 \cdot (\text{precision} \cdot \text{recall}) / (\text{precision} + \text{recall})$, summarizes both in one number and — unlike accuracy — stays honest under imbalance because it ignores the easy, abundant negatives.

6.3 Letting business cost choose the operating point

The metric is only half the decision; the other half is the **threshold**, and it should be set by the *asymmetry of the costs*, not left at the default 0.5.

Worked example. *Cancer screening from scans:* a false negative (missing a real tumor) can cost a life; a false positive (a needless follow-up scan) costs money and worry. The costs are

wildly asymmetric toward misses, so we tune the threshold for **high recall** even at the price of more false positives — better to re-examine ten healthy patients than to miss one tumor.

Spam filtering for email: a false positive (a real invoice sent to spam) may lose a customer; a false negative (one spam in the inbox) is a minor annoyance. Now the asymmetry runs the other way, so we tune for **high precision**.

The discipline is to state, before evaluating, which error is worse and by roughly how much, then choose the metric and threshold that reflect it. For multi-class problems with imbalance, prefer the **macro-averaged** F1 (average the per-class F1 so rare classes count as much as common ones) over the micro-average (which the majority class dominates).

Common pitfall Always report the metric on a **held-out test set** the model never saw, with the threshold fixed on the *validation* set. Tuning the threshold on the test set and then reporting that number is a subtle leak that makes the model look better than it is.

7 Object detection

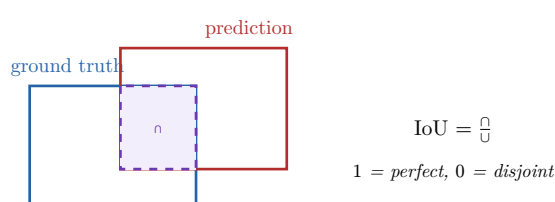
7.1 From “what” to “what and where”

Classification answers “is there a car in this image?” Detection answers “*where are* the cars, and how many?” — it returns, for each object, a **bounding box** (the rectangle enclosing it) and a class label with a confidence score. This is the difference between “this street scene contains pedestrians” and “there is a pedestrian at these pixels, stepping off the curb” — and for a self-driving car only the second is useful.

Modern detectors such as the **YOLO** family (“You Only Look Once”) run a single network pass over the image and predict many candidate boxes at once, which makes them fast enough for real-time video. We use them the way DAT62-2 taught us to use any pretrained model: load weights, run inference, interpret outputs — and fine-tune on our own boxes when needed.

7.2 IoU: measuring how good a box is

A predicted box is rarely pixel-perfect. To decide whether it is “correct” we need a way to score box overlap. **Intersection over Union (IoU)** is that score: the area where the predicted and ground-truth boxes overlap, divided by the area they jointly cover.

$$\text{IoU} = \frac{\text{area of intersection}}{\text{area of union}}.$$


$\text{IoU} = \frac{n}{U}$
1 = perfect, 0 = disjoint

Figure 10: IoU compares a prediction (red) to ground truth (blue). The shaded overlap divided by the total covered area gives a number in $[0, 1]$; a detection usually counts as correct above a threshold such as 0.5.

IoU does two jobs. At **evaluation** time, a detection is scored a true positive only if its IoU with a real object exceeds a threshold (commonly 0.5), so detection metrics like mean Average

Precision are IoU-based. At **inference** time, IoU is the tool that cleans up duplicate boxes — which brings us to NMS.

7.3 Non-maximum suppression

A single-pass detector typically fires *several* overlapping boxes on the same object — a cluster of rectangles all around one car, each with its own confidence. We want one box per object. **Non-maximum suppression (NMS)** is the simple greedy cleanup: keep the highest-confidence box, delete every other box that overlaps it too much (IoU above a threshold), and repeat with the remaining boxes.

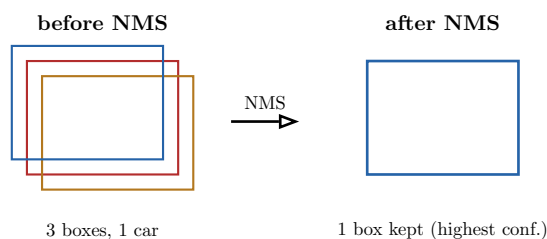


Figure 11: NMS collapses a cluster of overlapping detections into a single box — the most confident one — discarding its high-IoU neighbors as duplicates.

Common pitfall The NMS IoU threshold is a real trade-off, not a constant to ignore. Set it *too low* and NMS deletes legitimately distinct objects that happen to overlap — two pedestrians standing close get merged into one detection. Set it *too high* and duplicate boxes survive on a single object. When objects in your scenes crowd together, this threshold needs deliberate tuning.

8 A map of vision tasks

8.1 Three questions, three tasks

It is easy to reach for “computer vision” as one thing. In practice you must first decide *which* vision task your problem is, because the task dictates the labels you need, the model you pick, and the metric you report. Three tasks cover most of the field, distinguished by the granularity of their answer.

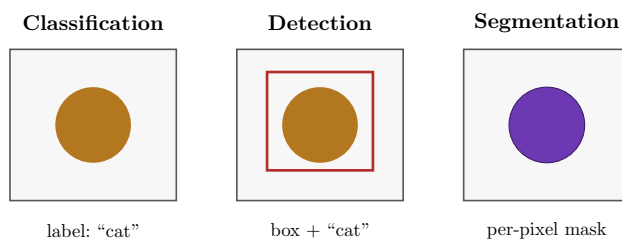


Figure 12: The granularity ladder. Classification labels the whole image; detection adds a box around each object; segmentation labels every pixel.

Image classification assigns one (or a few) labels to the whole image — “this scan shows pneumonia.” **Object detection** localizes each object with a box and a label — “three cars and one bus, here, here, here.” **Semantic segmentation** goes finer still, labeling *every pixel* with a class, so you get the exact shape of each region, not just a box.

8.2 One business use case each

Worked example. *Classification:* an insurer routes incoming claim photos by labeling each as “auto,” “property,” or “other,” so each goes to the right adjudication queue — a whole-image label is all that is needed.

Detection: a retailer audits shelf photos by detecting and counting each product’s box, flagging when a brand is out of stock — you must know *where* and *how many*, so a box per item is required.

Segmentation: a satellite-imagery firm measures deforestation by labeling every pixel as “forest” or “not forest,” because the *area* of each region is the deliverable and only a per-pixel mask gives it.

The labeling cost rises sharply down this ladder — a class label takes a second, a box a few seconds, a pixel mask minutes per image — so choosing the *least granular task that still answers the business question* is both a modeling and a budgeting decision.

9 Why production vision systems fail

9.1 Clean test scores lie

A model that hits 96% on your test set can fall to 60% in production, and the gap is not a bug in your code — it is a mismatch between the world your test set described and the world the model actually meets. Three failure modes account for most of these collapses. The professional skill is to *name* which one you face and pair it with a concrete mitigation.

9.2 Distribution shift

Distribution shift is when deployment data is drawn from a different distribution than your training data. The model has not “broken”; it is being asked about images unlike anything it learned from.

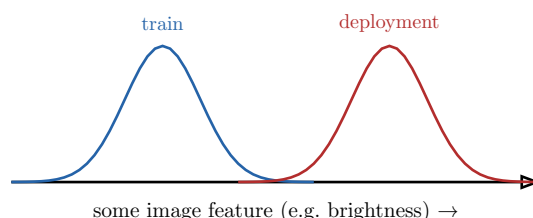


Figure 13: Distribution shift: the deployment data (red) is centered away from the training data (blue). Accuracy on the original test set says nothing about the region where the model now operates.

Worked example. A crop-disease classifier is trained on photos from one region’s farms, all shot in bright midday sun. Deployed in a rainier region where farmers photograph leaves under overcast skies, accuracy craters — same leaves, different lighting distribution. **Mitigations:** augment for the shift you expect (here, heavy brightness/contrast jitter); collect a sample of real deployment images and fine-tune on them; and *monitor* input statistics in production so you detect the shift instead of discovering it from angry users.

9.3 Label noise

Label noise is errors in your *training* labels — mislabeled images the model is told to trust. A model can only be as correct as its labels; train it on noise and it learns the noise.

Worked example. In a 200{,}000-image dataset labeled by a rushed crowd, perhaps 4% of “benign” skin lesions are actually malignant, mislabeled. The model dutifully learns to call some malignant lesions benign — the most dangerous possible error. **Mitigations:** audit a random sample to estimate the noise rate; use the model’s own high-confidence disagreements with the label to surface likely mistakes for re-labeling; require consensus among multiple annotators on hard cases; and prefer loss functions or training tricks that are robust to a known fraction of bad labels.

9.4 Long-tail class imbalance

Long-tail class imbalance is the common situation where a few classes are abundant and many classes are rare — a few “head” classes with thousands of examples and a long “tail” of classes with a handful each.

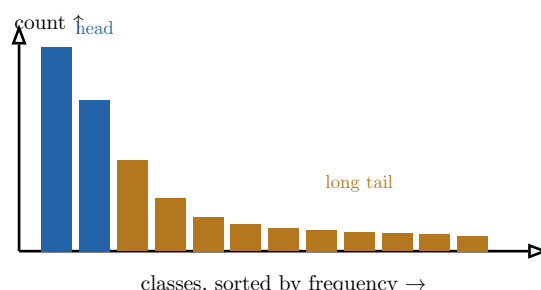


Figure 14: A long-tailed class distribution. A few head classes dominate the data; the model sees rare tail classes so seldom it never learns them well.

Worked example. A wildlife camera-trap classifier sees deer in 80% of images and a critically endangered cat in 0.1%. Trained naively it maximizes overall accuracy by almost always guessing common species and essentially ignores the rare cat — the one observation the ecologists actually care about. **Mitigations:** re-balance with oversampling of rare classes or class-weighted loss; augment the tail classes aggressively; group ultra-rare classes or use a hierarchy; and *evaluate with macro-averaged* metrics (the previous chapter) so tail performance is visible instead of being drowned out by the head.

Diagnosis checklist. Accuracy dropped in production? Ask: did the *inputs* change (distribution shift → augment, fine-tune on new data, monitor)? Are the *training labels* wrong (label noise → audit, re-label, consensus)? Are the *rare classes* failing while overall accuracy looks fine (long-tail → re-balance, macro metrics)? Naming the failure mode is half the fix.

10 Looking inside the model: Grad-CAM

10.1 Why a number is not enough

Your model says “malignant, 0.94 confidence.” Should the doctor trust it? Not yet — because the model might be right for the wrong reason. A famous failure: a pneumonia classifier reached high accuracy by reading the *hospital’s metadata stamp* in the corner of the X-ray (sicker patients came from one hospital), not the lungs. The number looked great; the model was useless. To catch this we need to see *where* the model looked.

10.2 What Grad-CAM shows

Grad-CAM (Gradient-weighted Class Activation Mapping) produces a heatmap over the input image showing which regions most increased the score for the predicted class. Intuitively, it asks the last convolutional layer — the one that still has spatial structure — “which of your feature locations, if turned up, would raise this class’s score the most?”, using the gradient of the class score with respect to each feature map as the importance weight. The result is a coarse, warm-where-it-mattered overlay.

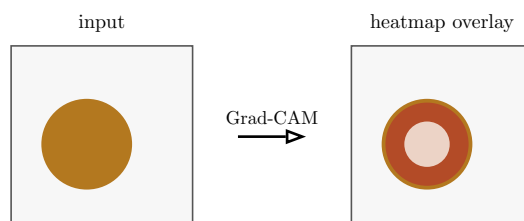


Figure 15: Grad-CAM overlays a heatmap on the input: warm regions are the pixels that most drove the predicted class. A model attending to the object (here, the animal) is trustworthy; one attending to a corner watermark is not.

10.3 Using it, and communicating it

Grad-CAM has two professional uses. As a **debugging** tool it reveals spurious correlations — if the heatmap lights up the background, a timestamp, or a ruler next to a tumor, your model has learned a shortcut and will fail in production. As a **trust** tool it lets a domain expert sanity-check individual predictions.

Worked example. Explaining a model to a non-technical stakeholder, you do not say “we backpropagate the class logit to the final feature maps and weight by the global average pooled gradients.” You say: “*The red regions are where the model was looking when it made this call. Here it focused on the lesion itself, which is what we’d want a clinician to look at — that’s a good sign. On this other image it focused on the ruler in the corner, which means we can’t trust that prediction and need to fix the data.*” You translate a heatmap into “where the model looked, and whether that’s sensible.”

Common pitfall Grad-CAM is deliberately *coarse* — it comes from the low-resolution final conv layer, so it points at a region, not exact pixels. Do not over-read a blob as a precise outline, and remember it explains *this one* prediction, not the model’s behavior in general. It is evidence, not proof.

11 Delivering a complete vision project

11.1 The arc, end to end

Everything in this course exists to support one capstone skill: shipping a computer vision project a stakeholder can trust. The earlier chapters were the parts; this is the assembly. A complete project moves through a predictable arc, and the discipline is to do each stage deliberately and record the decisions.

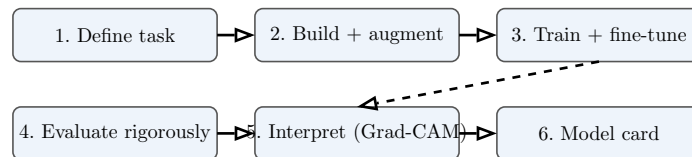


Figure 16: The project arc. Each stage maps to a chapter of this book; the model card at the end records the decisions made along the way.

1. Define the task.

State the business question, then pick the vision task (classification / detection / segmentation) that answers it at the lowest useful granularity, and choose the metric and operating point from the cost asymmetry — *before* looking at any model.

2. Build and augment the dataset.

Collect images that match deployment conditions, split into train/validation/test *before* any peeking, fix the preprocessing (resolution, color space, normalization), and design an augmentation pipeline whose every transformation you can justify by an invariance.

3. Train and fine-tune.

Start from a pretrained backbone; decide feature extraction vs fine-tuning from data size and domain gap; drive the training loop and read the loss/accuracy curves (DAT62-2) to catch over/underfitting early.

4. Evaluate rigorously.

Report the chosen metric on the untouched test set at the threshold fixed on validation; break performance down by class (macro-average) to expose long-tail weakness; probe for the production failure modes you can foresee.

5. Interpret.

Use Grad-CAM on a sample — especially on errors — to confirm the model attends to real signal, not shortcuts.

11.2 The model card

The deliverable that ties it together is the **model card** you met in DAT62-2 — a short, honest document so that someone who did not build the model can decide whether to trust it. For a vision project it states: the **task and intended use** the **data** (source, size, splits, known biases and collection conditions); the **preprocessing and augmentation** with justifications; the **architecture** and the feature-extraction-vs-fine-tuning choice with its rationale; the **evaluation** (metric, why that metric, score on test, per-class breakdown, chosen threshold); the **interpretation** evidence from Grad-CAM; and explicitly the **limitations and failure modes** — the distribution shifts, label-noise risks, and tail classes where the model should not be trusted.

Common pitfall The most common capstone failure is a model card that reports only the headline number and hides the weaknesses. A card that says “94% accuracy” and stops is worse than useless — it invites misplaced trust. A professional card names where the model *fails*: the lighting conditions it has not seen, the rare classes it gets wrong, the IoU threshold it is sensitive to. Honesty about limitations is the mark of the deliverable, not a blemish on it.

The whole course in one line. Get the pixels right (tensors, normalization), teach the right invariances (augmentation), reuse what exists (pretrained backbones), measure what matters (cost-aware metrics), localize when you must (detection, IoU, NMS), anticipate how it will break (shift, noise, long tail), prove where it looked (Grad-CAM), and write it all down honestly (model card).

Exercises

1. An RGB image of shape (3, 256, 256) is loaded with OpenCV and fed straight into an ImageNet ResNet, which then performs poorly. Name two distinct preprocessing bugs that could be responsible and how you would fix each.
2. For a model that detects cracks in bridge photos taken from any angle, decide whether each augmentation is justified and state the invariance it asserts: (a) horizontal flip, (b) 90° rotation, (c) vertical flip, (d) color jitter.
3. Explain, to a non-technical manager, why your fraud-image detector with 99.2% accuracy might still be unacceptable, and which metric you would report instead.
4. A detector outputs five overlapping boxes around one truck. Walk through how NMS reduces them to one, and describe a scene where a too-low NMS IoU threshold would cause a real error.
5. You have 1{, }200 X-ray weld images (grayscale, unlike ImageNet) and a pretrained ResNet. Choose feature extraction or fine-tuning, justify the choice on data size and domain gap, and name the main risk of your choice.
6. A deployed leaf-classifier's accuracy fell from 95% to 68% over winter. State which production failure mode you would investigate first, what evidence would confirm it, and one mitigation.